

7th Place Solution for the Vesuvius Challenge

Rank	7
Team	DECEM
Author	DECEM
Topic ID	679251
Version	Original

Source: <https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/discussion/679251>
Retrieved with Kaggle CLI on 2026-07-07.

Thanks to the organizers for putting together such a cool competition — learned a ton and had a lot of fun digging into volumetric topology.

Congrats to all the winners and participants! Here's a quick summary of what I did.

Overview

7th place, Public LB 0.618 / Private LB 0.591. Like many others, nnU-Net carried the day. Simple pipeline: two nnU-Net models ensembled + TTA + post-processing.

Model Training

Both models are based on **nnUNet** with `nnUNetPlannerResEncM`, `3d_lowres` configuration.

- Patch size: **128³**
- Epochs: **2000**
- Optimizer: **SGD** (nnUNet default: lr=0.01, momentum=0.99, nesterov=True, wd=3e-5)
- Loss: **CE Loss + Dice Loss + Skeleton Recall Loss**, all weighted equally (1.0 each)

The key difference between my two models:

Model A — Preprocessed training labels (key insight): I noticed that the organizers' GitHub repo has a specific processing pipeline for the test set data. I applied the **exact same processing to the training labels** before training. This was probably the single most impactful thing I did — the raw training labels don't quite match the test-time processing, and this mismatch hurts generalization. Aligning train/test label processing gave a clear and consistent boost.

Model B — Raw training labels: Standard training without any label preprocessing. This gives the ensemble some diversity.

Ensemble & Inference

- Simple ensemble of Model A + Model B
- **TTA** (nnUNet's built-in test-time augmentation with mirroring)

Nothing fancy here, just let nnUNet do its thing.

Post-Processing

A four-stage post-processing pipeline applied on binary predictions from nnU-Net to improve topological correctness, surface accuracy, and instance consistency.

Ridge Detection (Hessian-based Frangi Filter): The 3D Frangi sheetness filter is applied directly on the binarized prediction volume. It computes the Hessian matrix at each voxel via second-order Gaussian derivatives, then analyzes the sorted eigenvalues ($|\lambda_1| \leq |\lambda_2| \leq |\lambda_3|$) to compute a sheetness response. Voxels forming planar/sheet-like structures satisfy $|\lambda_1| \approx 0$, $|\lambda_2| \approx 0$, $|\lambda_3| \gg 0$, which corresponds to the papyrus

surface geometry. The response combines a background suppression term, a planarity term, and a blob rejection term. Only bright-on-dark structures ($\lambda_3 < 0$) are retained. The Frangi output is thresholded to produce a cleaned binary mask, effectively enhancing continuous surface regions while suppressing isolated noise points and non-planar false positives.

Coherence Enhancing Diffusion (CED): After ridge detection, Coherence Enhancing Diffusion is applied slice-by-slice (2D) on the Frangi-filtered volume to enhance spatial continuity along the papyrus surface. CED is a PDE-based anisotropic diffusion method governed by $\partial u / \partial t = \text{div}(D(J_p) \cdot \nabla u)$, where the diffusion tensor D is derived from the structure tensor J_p . The structure tensor is computed using Pavel Holoborodko derivative kernels with Gaussian smoothing. The diffusion tensor diffuses strongly along the dominant structure orientation (tangent to the papyrus surface) while preserving edges in the perpendicular direction, implemented with GPU acceleration via PyTorch. The result is re-binarized. This step fills small holes within surfaces, reconnects fragmented segments, and smooths boundary noise while maintaining edge sharpness.

Anisotropic Morphological Closing: A 3D binary closing operation (dilation followed by erosion) is applied with an anisotropic ellipsoidal structuring element. The Z-radius is set larger than the XY-radius, accounting for the fact that papyrus layers may have small gaps along the depth axis due to scanning artifacts. This fills remaining small holes and bridges tiny discontinuities that survived the CED step, without over-connecting adjacent layers in the XY plane.

Dust Removal (Small Object Pruning): Small connected components below a voxel-count threshold are removed from the final binary mask. This eliminates residual noise fragments and transient false positives that do not form meaningful papyrus structures, reducing spurious connected components.

Pipeline Summary

```
Binary Prediction (nnU-Net) + TTA
  → Frangi Sheetness Filter (3D)
  → CED Anisotropic Diffusion (2D slice-wise, GPU)
  → Anisotropic Closing
  → Dust Removal
  → Final Prediction
```

Note on Muon Optimizer

One thing worth mentioning — I experimented with the **Muon optimizer** as a replacement for SGD. Models trained with Muon consistently performed better on both Public and Private LB compared to SGD.

However, I didn't have enough compute to fully train all my final models with Muon, so my submitted ensemble still uses SGD-trained models. Training speed with SGD is only about **1.25x faster** than Muon, so it's not a huge tradeoff.

I've open-sourced my optimized Muon implementation: [FastMuon](#) — drop-in Turbo-Muon with Triton acceleration. If you have the hardware, worth trying.

What Didn't Work / Didn't Try

- More aggressive data augmentation — tried cranking up augmentation strengths / probabilities but it consistently hurt performance. Ended up using lower-than-default nnUNet augmentation probabilities
- cLDice Loss — didn't see improvement over the CE + Dice + Skeleton Recall combo
- Adding SE (Squeeze-and-Excitation) modules into the UNet — no meaningful gain, just extra VRAM usage
- Pseudo labels — tried using model predictions on unlabeled data as pseudo labels for semi-supervised training, but the noise was too much. Ended up degrading performance rather than helping
- More aggressive topological post-processing (PCA hole filling, Betti matching, etc.)

DECEM

Supplemental Q&A

Giorgio Angelotti · 2026-02-28T08:01:35.397000

Nice solution! I am also interested about your finding on the Muon optimizer. This is something we have been testing lately also for ink detection. Could you please share more?

DECEM · 2026-02-28T08:25:02.307000

Thanks! Here are my findings. I trained with the same config (nnUNetPlannerResEncM, 3d_lowres, 128³ patch, single fold, same post-processing) for 1000 epochs, only swapping the optimizer:

Optimizer	Implementation	Key Params	Private LB
SGD (nnUNet default)	<code>torch.optim.SGD</code>	<code>lr=0.01, momentum=0.99, nesterov=True, wd=3e-5</code>	0.590
AdamW	<code>torch.optim.AdamW</code>	<code>lr=3e-4, betas=(0.9, 0.999), eps=1e-8, wd=3e-5</code>	0.594
Muon+AdamW	<code>SingleDeviceMuonWithAuxAdam</code>	Muon (hidden weights, <code>ndim>=2</code>): <code>lr=3e-4, momentum=0.95, variant=turbo</code> ; AdamW (bias/norm/seg_head): <code>lr=3e-4, betas=(0.9, 0.95), eps=1e-10, wd=0.01</code>	0.595

The Muon variant uses `SingleDeviceMuonWithAuxAdam` from [FastMuon](#) — Turbo-mode (AOL preconditioning + 4 Newton-Schulz iterations) is applied to conv/linear weight matrices (`ndim>=2`, excluding embed/head/seg_layers), while the built-in AdamW handles bias, normalization, and segmentation head parameters. All use PolyLR scheduling.

SGD training is only ~1.25x faster than Muon, so the speed tradeoff is small.

Note: I didn't have enough compute to fully explore Muon's hyperparameter space (e.g. higher lr like 0.02 as originally recommended). With proper tuning the gap could be larger.